Technical Report: Q-Learning on FrozenLake-v1

Topics in AI | Assignment I (CLO-I) | 20 October 2026

1. Problem Description

FrozenLake-v1 is a 4x4 grid with start (S), safe frozen cells (F), holes (H), and goal (G). The agent starts at state 0 and aims to reach state 15 without entering a hole. There are 16 discrete states (0–15) and four discrete actions: left (0), down (1), right (2), up (3). Reward is +1 for reaching the goal and 0 otherwise. Entering a hole or reaching the goal ends an episode. We set is_slippery=False, so actions are deterministic. This choice makes tabular Q-learning easy to explain and reproduce.

2. Methodology

The agent maintains a 16 x 4 Q-table, initialized to zero. During training it chooses a random action with probability epsilon, otherwise the action with the highest Q-value. After each transition it applies:

Q(s,a) ← Q(s,a) + alpha [r + gamma max Q(s',a') - Q(s,a)]. For terminal transitions, the future-value term is zero. Epsilon decays each episode down to a fixed minimum. Training runs for 15,000 episodes, each limited to 100 steps. Evaluation uses a greedy policy with no exploration.

3. Experimental Setup and Hyperparameters

Parameter	Value
Environment	FrozenLake-v1, 4x4, non-slippery
Algorithm	Tabular Q-learning
Episodes / max steps	15,000 / 100
Learning rate (alpha)	0.8
Discount (gamma)	0.95
Epsilon initial / minimum	1.0 / 0.05
Epsilon decay	0.9995 per episode
Random seed	42

4. Results and Discussion

A local reference run using equivalent deterministic FrozenLake transition rules reached the goal in its greedy demonstration (6 steps) and achieved 94% success over the last 200 training episodes. These are reference results, not a verified Gymnasium execution; run main.py with Gymnasium to confirm final metrics and notebook outputs. The figure shows the 200-episode moving success rate.



The deterministic environment permits reliable convergence after sufficient exploration. Failures occur when exploratory actions enter holes; the policy can also fail early in training before Q-values propagate from the goal. This implementation does not establish performance on the stochastic (slippery) variant. A useful comparison is early vs late training: the moving success rate rises as the agent learns. Hyperparameter sensitivity: lower epsilon decay explores longer, while a higher gamma gives more importance to delayed rewards; these are conceptual expectations, not separate measured sweeps.

5. Conclusion

A from-scratch Q-learning agent learned a goal-reaching policy for deterministic FrozenLake. The submission includes Python modules, a Q-table, a training curve, a notebook and an animated policy demonstration. Future work should compare several learning rates and test the slippery environment.